

Table Design – Distribution Styles in Amazon Redshift

To Begin with the Lab

Summary of the Lab

In this lab, different distribution styles in **Amazon Redshift** were explored to understand how data is physically distributed across slices in a cluster. The existing users table was analyzed to identify its distribution key and distribution style, and storage distribution was examined using Redshift system views. New tables were then created using KEY, EVEN, ALL, and AUTO distribution styles to compare their behavior and impact on slice-level data balancing. Disk usage statistics were reviewed after each table creation to observe how rows were distributed across slices. Finally, the results were verified to understand which distribution style is best suited for different workloads and how proper distribution improves query performance and storage efficiency.

Understanding Distribution Styles in Amazon Redshift

Amazon Redshift distributes table data across slices so that multiple parts of a query can run in parallel. The goal is to spread data evenly and reduce the amount of data that must be moved between nodes during joins and aggregations. Choosing the wrong distribution style can create data skew, where some slices store significantly more rows than others, causing slower query execution. Different distribution styles exist because different types of tables require different storage strategies based on size, update frequency, and query patterns.

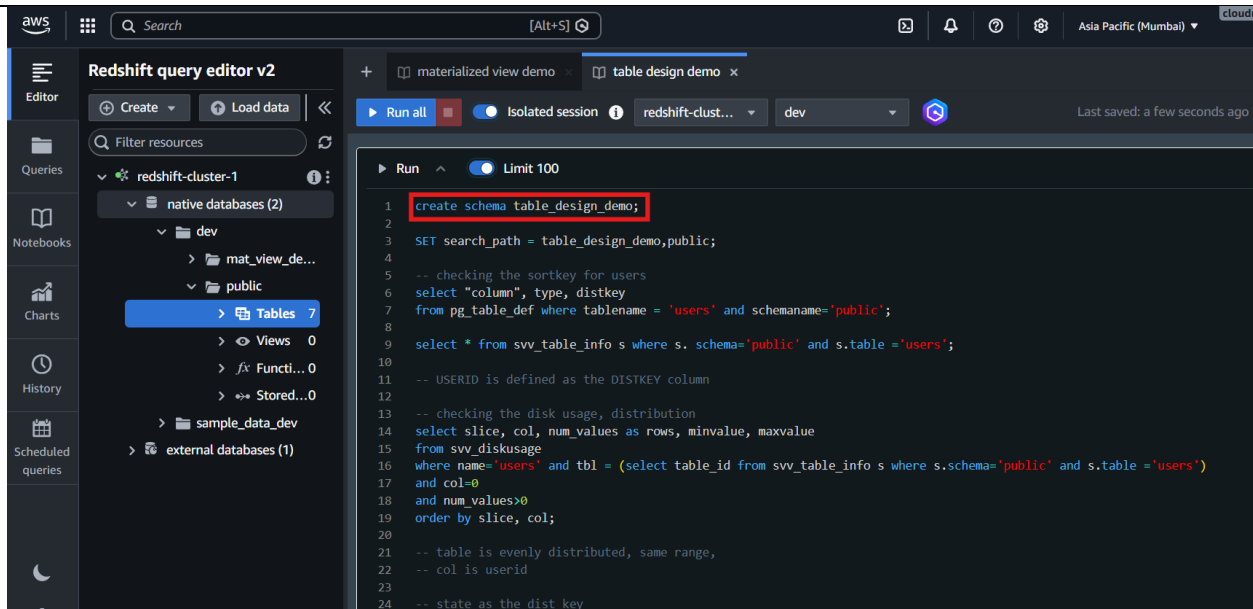
Example:

Consider an e-commerce system with an Orders table containing 50 million records and a Customers table containing 1 million records. If both tables use customer_id as the distribution key, matching customer records are stored on the same slices and joins execute faster because less data movement is required. However, if a low-cardinality column such as state is used instead, most records may accumulate on a few slices, resulting in uneven storage and slower query performance.

Lab Steps

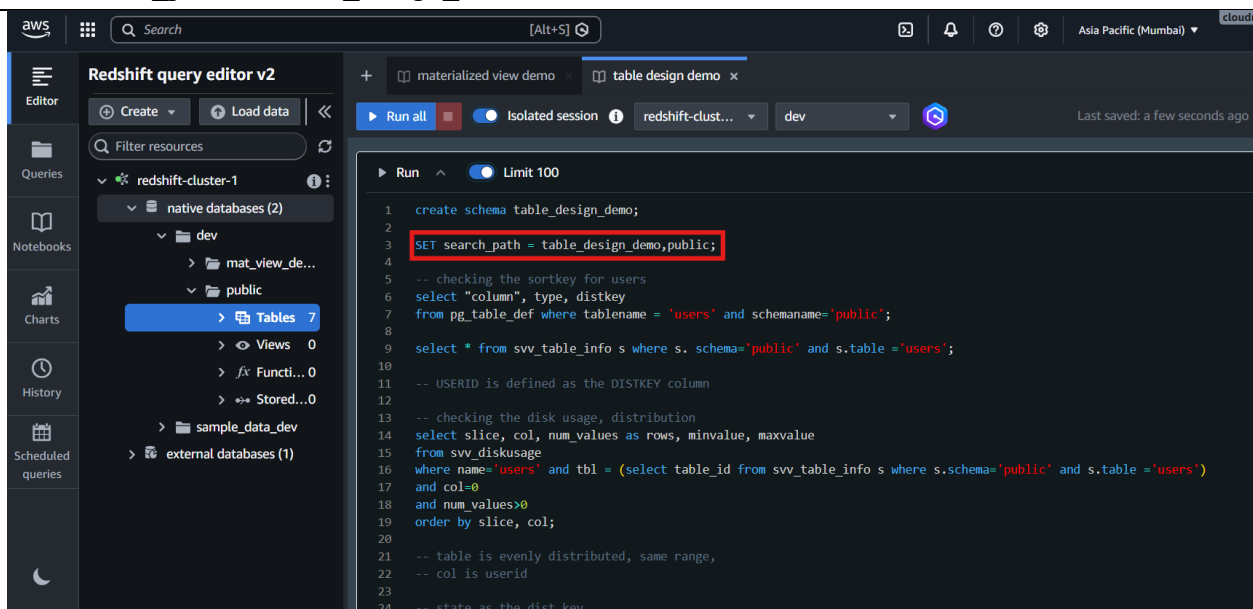
- Sign in to the AWS Management Console
- Navigate to **Amazon Redshift** → open **Redshift Query Editor v2**
- Create a schema for the lab

CREATE SCHEMA table_design_demo;



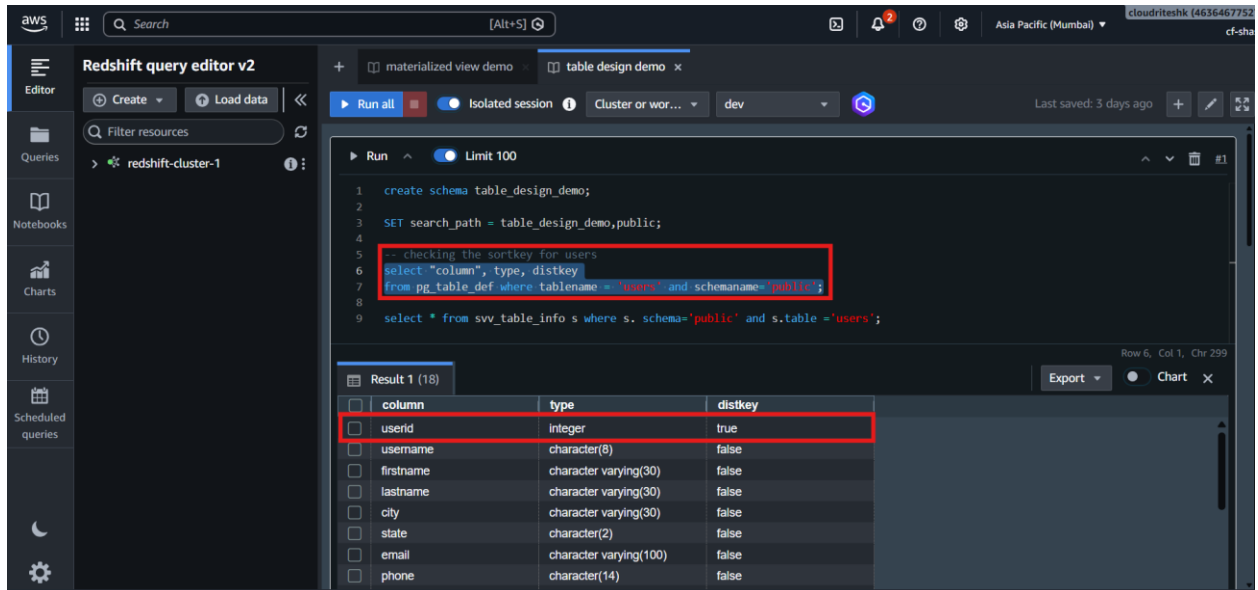
- Set the schema search path

SET search_path TO table_design_demo;



- Check the distribution key of the users table

```
SELECT tablename,
       "column",
       distkey
FROM pg_table_def
WHERE schemaname = 'public'
AND tablename = 'users';
```



The screenshot shows the AWS Redshift Query Editor v2 interface. The query editor displays the following SQL code:

```
1 create schema table_design_demo;
2
3 SET search_path = table_design_demo,public;
4
5 -- checking the sortkey for users
6 select "column", type, distkey
7 from pg_table_def where tablename = 'users' and schemaname = 'public';
8
9 select * from svv_table_info s where s.schema='public' and s.table = 'users';
```

The query results are displayed in a table with 3 columns: column, type, and distkey. The results are as follows:

column	type	distkey
userid	integer	true
username	character(8)	false
firstname	character varying(30)	false
lastname	character varying(30)	false
city	character varying(30)	false
state	character(2)	false
email	character varying(100)	false
phone	character(14)	false

- Verify that userid is configured as the distribution key
- Understand that rows are distributed across slices using userid
- Check the distribution style of the users table

```
SELECT table,
       diststyle,
       skew_rows
FROM svv_table_info
WHERE table = 'users';
```

```

1 create schema table_design_demo;
2
3 SET search_path = table_design_demo,public;
4
5 -- checking the sortkey for users
6 select "column", type, distkey
7 from pg_table_def where tablename = 'users' and schemaname='public';
8
9 select * from svv_table_info s where s.schema='public' and s.table='users';

```

schema	table_id	table	encoded	diststyle
public	107134	users	Y, AUTO(ENCODE)	KEY(userid)

- Verify that the users table uses KEY distribution
- Verify that userid is the active distribution key
- Check slice-level storage distribution

```

SELECT slice,
       num_values,
       minvalue,
       maxvalue
FROM svv_diskusage
WHERE name = 'users';

```

```

1 -- USERID is defined as the DISTKEY column
2
3 -- checking the disk usage distribution
4 select slice, col, num_values as rows, minvalue, maxvalue
5 from svv_diskusage
6 where name='users' and tbl = (select table_id from svv_table_info s where s.schema='public' and s.table='users')
7 and col=0
8 and num_values>0
9 order by slice, col;

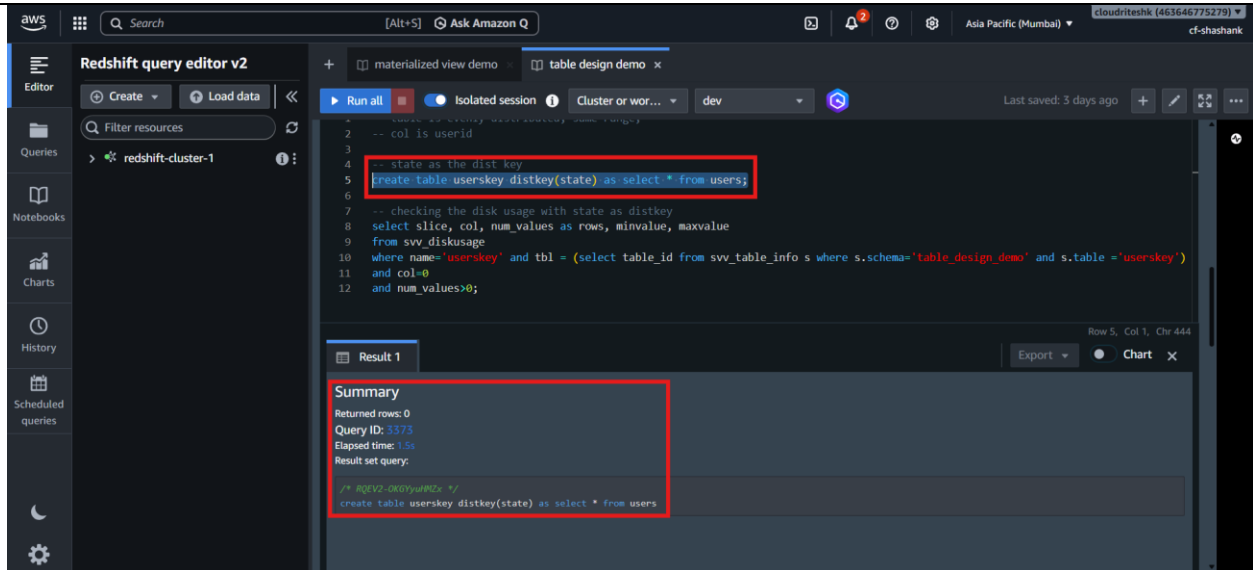
```

slice	col	rows	minvalue	maxvalue
0	0	24958	2	49990
1	0	25032	1	49988

- Verify that rows are distributed almost equally across slices
- Verify that the cluster contains one node with two slices

- Confirm that userid distributes data efficiently
- Create a table using state as the distribution key

```
CREATE TABLE userskey
DISTSTYLE KEY
DISTKEY(state)
AS
SELECT * FROM public.users;
```



- Check disk usage for the userskey table

```
SELECT slice,
       num_values,
       minvalue,
       maxvalue
FROM svv_diskusage
WHERE name = 'userskey';
```

The screenshot shows the AWS Redshift Query Editor v2 interface. The query editor contains the following SQL code:

```

4 -- state as the dist key
5 create table userskey distkey(state) as select * from users;
6
7 -- checking the disk usage with state as distkey
8 select slice, col, num_values as rows, minvalue, maxvalue
9 from svv_diskusage
10 where name='userskey' and tbl = (select table_id from svv_table_info s where s.schema='table_design_demo' and s.table='userskey')
11 and col=0
12 and num_values>0;

```

The results of the second query are displayed in a table:

slice	col	rows	minvalue	maxvalue
0	0	30368	1	49990
1	0	19622	4	49987

Query ID: 3461 Elapsed time: 877 ms Total rows: 2

- Observe that rows are distributed unevenly across slices
- Verify that one slice contains significantly more rows than the other
- Confirm that state is not an effective distribution key for this dataset
- Create a table using EVEN distribution

```

CREATE TABLE userseven
DISTSTYLE EVEN
AS
SELECT * FROM public.users;

```

The screenshot shows the AWS Redshift Query Editor v2 interface. The query editor contains the following SQL code:

```

1 -- not evenly distributed
2 -- even e. g.
3 create table userseven diststyle even as
4 select * from users;
5
6 select slice, col, num_values as rows, minvalue, maxvalue
7 from svv_diskusage
8 where name='userseven' and tbl = (select table_id from svv_table_info s where s.schema='table_design_demo' and s.table='userseven')
9 and col=0
10 and num_values>0;

```

The results of the second query are displayed in a table:

slice	col	rows	minvalue	maxvalue
0	0	30368	1	49990
1	0	19622	4	49987

Summary

Returned rows: 0
 Query ID: 3484
 Elapsed time: 1.1s
 Result set query:

```

/* RQE_V2-r0b5bhy00H */
create table userseven diststyle even as
select * from users

```

- Check disk usage for the userseven table

```

SELECT slice,
       num_values

```

```
FROM svv_diskusage
WHERE name = 'userseven';
```

The screenshot shows the AWS Redshift Query Editor v2 interface. The query editor displays the following SQL code:

```
1 -- not evenly distributed
2 -- even e.g.
3 create table usersseven diststyle even as
4 select * from users;
5
6 select slice, col, num_values as rows, minvalue, maxvalue
7 from svv_diskusage
8 where name='userseven' and tbl = (select table_id from svv_table_info s where s.schema='table_design_demo' and s.table = 'usersseven')
9 and col=0
10 and num_values>0;
```

The results table, titled 'Result 1 (2)', shows the following data:

slice	col	rows	minvalue	maxvalue
0	0	25000	2	49920
1	0	24990	1	49990

- Verify that rows are distributed almost equally across slices
- Confirm that EVEN distribution uses a round-robin allocation method
- Create a table using ALL distribution

```
CREATE TABLE usersall
DISTSTYLE ALL
AS
SELECT * FROM public.users;
```

The screenshot shows the AWS Redshift Query Editor v2 interface. The query editor displays the following SQL code:

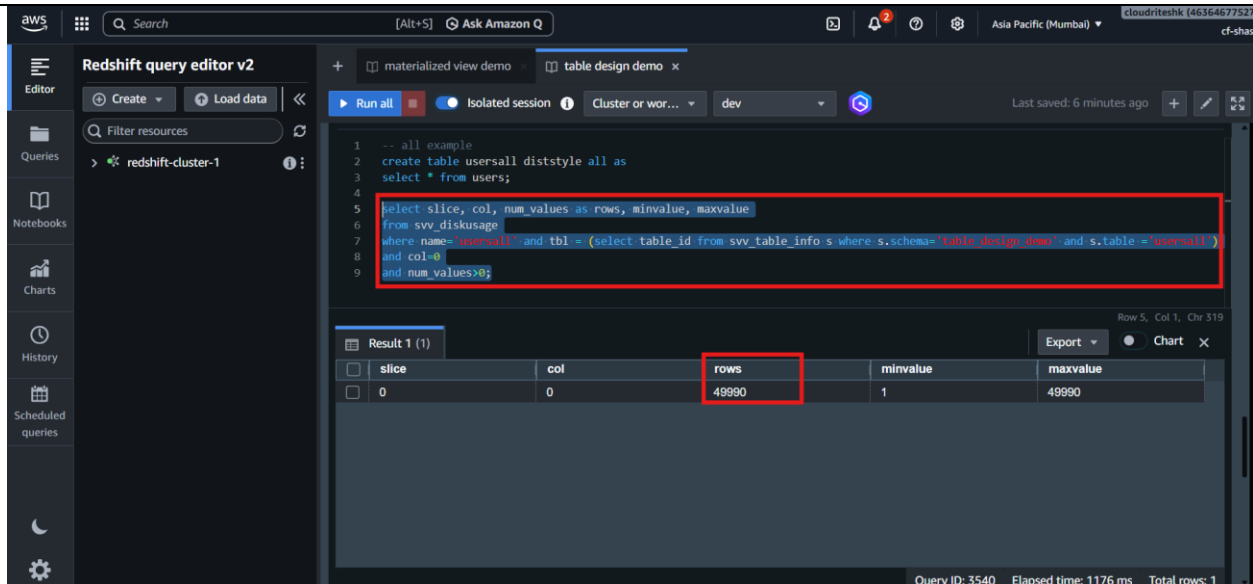
```
1 -- all example
2 create table usersall diststyle all as
3 select * from users;
4
5 select slice, col, num_values as rows, minvalue, maxvalue
6 from svv_diskusage
7 where name='usersall' and tbl = (select table_id from svv_table_info s where s.schema='table_design_demo' and s.table = 'usersall')
8 and col=0
9 and num_values>0;
```

The results table, titled 'Result 1', shows a summary of the query execution:

Summary
Returned rows: 0
Query ID: 1525
Elapsed time: 1.5s
Result set query:

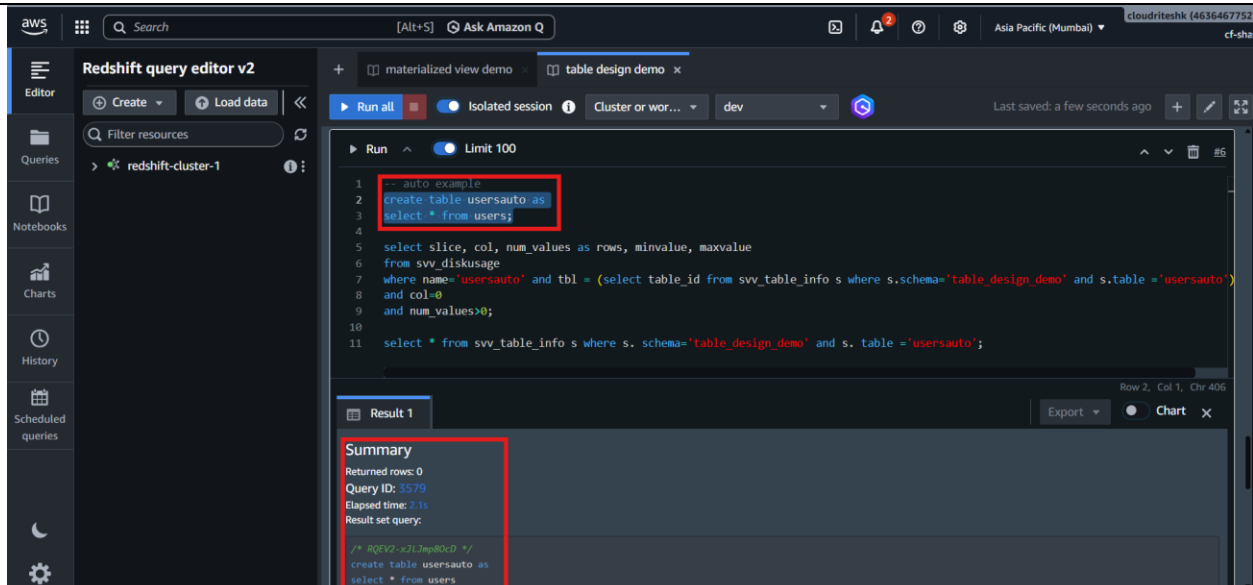
- Check disk usage for the usersall table

```
SELECT *
FROM svv_diskusage
WHERE name = 'usersall';
```



- Verify that the complete table is copied to every node
- Confirm that no slice-level distribution occurs
- Understand that ALL distribution is best suited for small lookup tables that are rarely updated
- Create a table using AUTO distribution

```
CREATE TABLE usersauto
AS
SELECT * FROM public.users;
```



- Check the automatically selected distribution style

```
SELECT table,  
       diststyle  
FROM svv_table_info  
WHERE table = 'usersauto';
```

The screenshot shows the Amazon Redshift Query Editor v2 interface. The SQL query in the editor is as follows:

```
1 -- auto example  
2 create table usersauto as  
3 select * from users;  
4  
5 select slice, col, num_values as rows, minvalue, maxvalue  
6 from svv_diskusage  
7 where name='usersauto' and tbl = (select table_id from svv_table_info s where s.schema='table_design_demo' and s.table='usersauto')  
8 and col=0  
9 and num_values>0;  
10  
11 select * from svv_table_info s where s.schema='table_design_demo' and s.table='usersauto';
```

The results of the query are displayed in a table with the following columns: slice, col, rows, minvalue, and maxvalue. The 'rows' column is highlighted with a red box.

slice	col	rows	minvalue	maxvalue
0	0	24958	2	49990
1	0	25032	1	49988

- Verify that **Amazon Redshift** automatically selects KEY distribution
- Verify that userid is selected as the distribution key
- Verify that KEY distribution is suitable for join-heavy workloads
- Verify that EVEN distribution provides balanced storage through round-robin allocation
- Verify that ALL distribution is suitable for small lookup tables
- Verify that AUTO distribution allows **Amazon Redshift** to choose the most appropriate distribution strategy automatically
- Verify that distribution style directly impacts storage balance, data movement, and query performance in **Amazon Redshift**